



.io

Bridging Blockchain and Off-Chain Compute

Making Hard Things Easy In Web3

Version 0.1

Audie Sheridan, Ben Murray, David Post

June 5, 2024

Contents

1	Introduction	4
2	Problem Statement	5
2.1	Dependence on Centralized Services	5
2.2	Technical Difficulties	6
2.3	Business Challenges	6
3	W3.io Solution	7
4	Ecosystem and Partnerships	8
4.1	Building a Strong Network	8
4.2	Knowledge and Integration	9
4.3	Transparency and Accessibility	9
4.4	Synergistic Benefits	9
4.5	Conclusion	9
5	Partner Usage In Protocol	10
6	User Experience (UX)	11
6.1	CLI	12
6.2	Visual	12
7	Ingredients	12
8	Recipes	13
9	W3.io Protocol	14
9.1	Network	15
9.2	Nodes	15
9.3	Types of Nodes	17
9.3.1	High Availability Nodes	18
9.3.2	Normal Nodes	18
9.3.3	Community Nodes	18
9.3.4	Compute Execution	18
9.3.5	Verifiability	19
9.3.6	Consensus	21
9.3.7	Trusted Execution Environments (TEEs)	22
9.4	Protocol Services	22
9.4.1	API	22
9.4.2	UI	23
9.4.3	Decentralized Task Queue	23
10	Token Model (Light Overview)	23
11	Business Model	23

12 Conclusion	23
References	24
A Network Partners	25
B Team Profiles	26
C Appendix C: Mathematical Models	27

DRAFT

Abstract

W3.io introduces a protocol aimed at bridging the gap between blockchain's on-chain execution capabilities and the computational power of the many decentralized off-chain compute projects in web3. This integration tackles real-world business challenges by enabling seamless orchestration of diverse on and off-chain compute tasks across a network of decentralized nodes. Despite the proliferation of decentralized solutions addressing various computational needs, a significant gap remains in integrating these solutions to address complex business processes effectively. W3.io proposes a decentralized network where nodes are incentivized to execute compute tasks, acting as the glue that integrates on-chain and off-chain computations. Through customizable recipes and ingredients, users can define and execute business processes with unprecedented flexibility and efficiency. The initiative is poised to redefine how businesses interact with blockchain ecosystems, offering a more integrated, efficient, and scalable approach to solving real-world problems. This enhances the capabilities of existing decentralized applications and opens new avenues for innovation and collaboration within the web3 space.

1 Introduction

Blockchain tech is a revolution in how we handle digital transactions and build decentralized apps, promising security and transparency. But it's not a magic bullet—it's got its limits. Things like slow transaction speeds, high costs, and limited on-chain power mean it can't do everything on its own, so we take much of the work off-chain. In the past few years, many decentralized projects launched that can replace small parts of IT offers with more efficient decentralized options.

There is a proliferation in individual chains. Some projects strive to be cross chain, others launch their own special purpose chains. It's become difficult to keep up, and the immense value web3 off-chain compute projects bring remains hidden and difficult to access. Web3 is both difficult and vast.

W3.io emerges as a pioneering solution in this landscape, designed to bridge the gap between the on-chain capabilities of blockchain technologies and the computational power of off-chain systems. This whitepaper introduces the W3.io protocol, a decentralized framework that facilitates seamless orchestration of on-chain and off-chain compute tasks. By leveraging a network of decentralized nodes, W3.io enables businesses to execute complex, integrated workflows with enhanced efficiency and flexibility.

The need for such a solution is driven by several critical challenges currently faced by developers and enterprises in the web3 space. Firstly, the over-reliance on centralized services contradicts the decentralized ethos of blockchain, introducing risks such as single points of failure and potential censorship. Secondly, the technical complexities involved in integrating disparate IT systems can stifle innovation and escalate costs. Lastly, the business challenges of deploying decentralized solutions—such as legal intricacies, payment issues, and the steep learning curve associated with new technologies—can deter enterprises from adopting blockchain technology.

W3.io addresses these challenges by providing a decentralized network of nodes that acts as an integrative backbone for . The protocol supports customizable "recipes" that define workflows triggered by user-defined events, progressing through a series of verifiable operations executed by the network nodes. This approach not only democratizes access to blockchain technology but also enhances the operational capabilities of existing decentralized applications, paving the way for innovative, complex, decentralized solutions.

This whitepaper details the architecture, operation, and potential applications of the W3.io network, illustrating how it can transform the landscape of blockchain integration and foster a new wave of decentralized innovation. By reducing dependency on centralized infrastructures, simplifying integration challenges, and addressing key business hurdles, W3.io is poised to play a crucial role in the evolution of blockchain ecosystems.

2 Problem Statement

When grappling with the world of web3, developers and businesses face a trio of challenges that can often slip under the radar in the rush to innovate. These hurdles include a heavy reliance on centralized services, the technical headaches of integration, and the business complexities of deploying complex decentralized solutions.

2.1 Dependence on Centralized Services

On the business front, the main priority often boils down to speed of delivery (Elliot, 2014), which naturally pushes developers towards the ready-to-go, easy-to-use centralized solutions. This not only shifts on-chain value off-chain but also racks up about \$180 million annually spent by web3 projects on centralized compute.**[CITATION NEEDED]** The convenience of centralized compute services like Amazon Web Services, Google Cloud, and Microsoft Azure is undeniable. They make spinning up new servers, containers or functions a breeze, which is why they're so attractive to web3 developers. However, this reliance is at odds with the decentralized ethos of web3 and brings with it several significant drawbacks:

- **Centralization Risks:** Using centralized services introduces single points of failure and potential censorship or control by central authorities, which goes against the grain of web3's decentralized promise. (Nakamoto, 2008)
- **Data Privacy Concerns:** There's always a risk that data processed on centralized platforms could be accessed or misused, posing serious privacy risks for sensitive blockchain applications. (Zyskind et al., 2015)
- **Vendor Lock-in:** Being tied to specific cloud providers can lead to vendor lock-in, making it tough and expensive to switch services or operate in a truly decentralized fashion. (Armbrust et al., 2010)
- **Cost Predictability:** The costs of using centralized compute services can be unpredictable and may spiral as operations grow, which is a challenge for projects on tight budgets. (Vaquero & Roderio-Merino, 2014)
- **Network Latency:** Relying on centralized servers can introduce latency issues, especially when the blockchain network is spread worldwide but the compute resources are centralized in specific locations. (Bonomi et al., 2012)
- **Regulatory Exposure:** Centralized services are subject to the laws and regulations of the places they operate in, which can bring additional constraints and risks not present in a decentralized setup. (Lessig, 1999, see Chapters 7 and 10)

2.2 Technical Difficulties

Piecing together various parts of the IT stack in a decentralized way to solve a complex business problem is currently quite a challenge. Starting such an integration requires not only hiring pricey web3 development talent but often also involves that talent firing up centralized compute to build and test everything piece by disconnected piece. The lack of a common agreement on which blockchain to use only adds to the testing and development problems. Different decentralized solution providers may have dramatically different ways of working with blockchain technology.

Developers working in web3 often find themselves doing tedious tasks during integration type projects. Things like translating data, simple data storage, devops, IT administration, and any other of a host of things required to make the customer project work, but that they've also done a hundred times before. Most web3 development is bespoke, and consists of easily repeatable patterns. This presents multiple risks for the customers:

- **High Development Costs:** Web3 development is obscenely expensive, and you don't pay less for hours spent solving tedious work orthogonal to the crux of your business problem.
- **Suboptimal Integrations:** Developers hired by you to develop your solutions will not be expert on every integration you need, so the quality of those integrations will vary widely
- **Chance of Failure:** Software development projects often fail, and complexity multiplies the chances of that. The less code you need to solve your business problem, the better.
- **Brittle Solutions:** Your solution may ultimately succeed, but a bespoke implementation will require specialized maintenance. The availability of your developer in a year or two, when the solution needs an upgrade or bug fix, is uncertain. Reliability in a crisis is also not guaranteed.

2.3 Business Challenges

The task of using multiple decentralized services together to tackle business needs isn't just technically demanding—it's also time-consuming and costly. In order to use multiple off-chain computation providers to solve your business problems, you have to know them well. You have to be technically competent about their solution and how to use it, you have to arrange terms, engage legal services perhaps, and most importantly, perhaps, you have to *wait* until these challenges are finished to launch the actual solution you're after.

In addition to the hurdles in place if you already know the decentralized solution providers you want to use, there is the problem of even being aware a solution you could use exists. There are hundreds or thousands of decentralized projects, each with their own unique perspective. They vary in how well they

tell you *what* they do and *how* they do it, but most are bad at describing *why* you should be interested. Just knowing the possibilities presents its own challenge.

Assuming a customer were able to navigate all the above business challenges, then comes the problem of payment. With a multitude of partners, chains, and other providers, coordinating proper payment will be a significant barrier. Some providers will want payment at time of service, others as a subscription (overbuying for your need), blockchains require gas or fees to execute, partner payments might require holding tokens or just paying in specific tokens. All of this adds up to a significant headache for even a slightly complex business process.

3 W3.io Solution

W3.io directly addresses the trio of challenges identified in the Problem Statement—dependence on centralized services, technical integration difficulties, and complex business deployments—by establishing a decentralized network that acts as the integrative backbone for various IT verticals and other solutions provided by our network partners. This network not only supports blockchain operations but also seamlessly ties together services from over 40 decentralized compute partners, making it a robust solution for real-world business applications.

- **Decentralized Network as a Solution to Centralization:** Our decentralized network mitigates the risks associated with centralization, such as single points of failure and potential censorship, by distributing compute tasks across an array of decentralized nodes. This structure ensures enhanced data privacy and reduces regulatory exposure, aligning with the decentralized ethos of web3 and improving cost predictability.
- **Integration of Decentralized Compute Partners:** The core of W3.io's functionality lies in its ability to orchestrate complex compute tasks through customizable 'recipes.' These recipes define workflows that begin with a customer-defined trigger and progresses through a series of decentralized operations. Each step in a recipe is executed verifiably by either a decentralized compute partner, or by the node in w3.io that interacts with the blockchain or external provider needed. This ensures the integration of multiple decentralized partners is seamless and efficient.
- **Addressing Technical and Business Challenges:** By leveraging the specialized capabilities of our decentralized partners, W3.io addresses the high costs and technical challenges associated with web3 development. Each partner can act as an 'ingredient' in our recipes, allowing us to provide a broad IT solution that is both repeatable and testable. This approach significantly reduces the redundancy and inefficiency typically involved in bespoke web3 projects, thereby lowering development costs and enhancing project viability.
- **Pre-built Network for Rapid Deployment:** W3.io has pre-established agreements with a diverse

range of decentralized compute partners, enabling rapid deployment of decentralized solutions. This ready-to-deploy framework is crucial for businesses looking to integrate complex systems without the need for extensive legal or technical groundwork. Our model simplifies the process of using multiple decentralized services, thereby solving the business challenge of lengthy and costly integrations.

- **Unified Payments:** Due to the established agreements with the decentralized compute partners, w3.io will consolidate the payment needed for these partners. Instead of separately paying several partners, any involved chain's fees, or other integrated provider, w3.io will provide the option of paying once for the entire workflow. This will provide a means to abstract the payment from the underlying complexity, which w3.io will navigate on behalf of the customer.

W3.io provides a comprehensive solution that not only addresses the inherent challenges of web3 development but also enhances the capabilities of blockchain technology by making it easy to integrate it with off-chain compute resources. Through our network of decentralized partners and our decentralized node network running customizable recipes, we offer businesses a scalable, efficient, and secure platform to execute their operations, paving the way for innovative applications in the web3 space.

4 Ecosystem and Partnerships

[NEEDS DAVID AND BEN COPYWRITING AND EDITING]

One of the most important aspects of how W3.io builds value is in the strength of the network of decentralized compute providers and chains that want to be a part of this solution. The effort to assemble over 40 partners into this network was a massive, 2 year undertaking, and reflects our commitment to a broad and effective ecosystem.

4.1 Building a Strong Network

The establishment of our partner network is not just about quantity but also about the quality and synergy between the partners. Each partner has been carefully selected based on their commitment to the vision of decentralized solutions and their ability to solve specific business problems. This pre-established network not only saves our customers the effort of vetting potential service providers but also ensures that they have access to top-tier solutions across the blockchain and IT spectrum.

4.2 Knowledge and Integration

We understand our partners' capabilities deeply and bring this knowledge to bear for the benefit of our customers. By building integrations in cooperation with our partners, we ensure that these integrations are robust, effective, and tailored to real-world applications. This collaborative approach enhances the functionality and reliability of our ecosystem, providing our customers with seamless access to a comprehensive suite of services.

4.3 Transparency and Accessibility

In our network, each partner is exposed in a way that highlights their strengths and unique offerings. We make it clear why customers should choose these solutions, aiding in informed decision-making. Importantly, W3.io is not rent-extractive towards our partners; we do not charge them for participation in our network. Instead, our model is designed to benefit our partners by exposing them to a broader customer base, thereby enhancing their business development opportunities.

4.4 Synergistic Benefits

The collaborative environment of our network means that every partner's business development efforts indirectly benefit all other partners. Since each partner typically offers only a part of the complete IT/Blockchain solution a customer might need, our platform facilitates the integration of these various services into cohesive solutions. This is achieved without the need for complex business agreements directly between multiple partners, as W3.io provides a unified framework with rules that are flexible enough to accommodate diverse solutions yet strict enough to ensure effective problem-solving.

4.5 Conclusion

The ecosystem of partners within W3.io is a cornerstone of our strategy to provide decentralized, integrated solutions that are more than the sum of their parts. By fostering a network where collaboration and mutual benefit are key, we not only enhance the capabilities of individual partners but also create a more robust and versatile platform for our customers. This network-driven approach is pivotal in re-defining how businesses interact with blockchain technologies and decentralized applications, paving the way for innovative solutions in the web3 space.

5 Partner Usage In Protocol

In the development and operation of the W3.io protocol, the integration of our partners' solutions is not just a feature—it's a fundamental aspect of our architecture. By incorporating our partners directly into the core of our protocol, we demonstrate the same trust and reliance on decentralized solutions that we advocate to our customers. This approach not only enhances the robustness and capabilities of the W3.io network but also serves as a live case study for the potential and effectiveness of decentralized compute resources.

Strategic Integration of Partners

Our protocol leverages the specialized services offered by our partners to address internal operational needs, mirroring the integration challenges our customers face. This strategic use of partners' technologies within our core operations serves multiple purposes:

- **Proof of Concept:** By using our partners' solutions to solve our own challenges, we provide a tangible proof of concept that showcases the practical applications and benefits of these technologies. This not only builds confidence in our protocol but also demonstrates the real-world utility of our partners' offerings.
- **Enhanced Credibility:** Integrating our partners deeply into our protocol operations enhances the credibility of our network. It shows that we stand behind the decentralized ethos we promote, committing to the same solutions we recommend to our users.
- **Continuous Improvement and Feedback:** This close integration allows us to continuously test, evaluate, and provide feedback on the technologies we use. It creates a feedback loop that benefits both W3.io and our partners, leading to improvements and optimizations in the solutions offered.

Transparency and Choice

Transparency is key in decentralized systems, and at W3.io, we commit to maintaining a high level of transparency regarding which partners we integrate and why. By publishing our choices and the rationale behind them, we aim to:

- **Educate Our Users:** Help our users understand why certain decentralized solutions are chosen over others and how they can be leveraged effectively in different scenarios.

- **Promote Informed Decision-Making:** By sharing our experiences and insights, we enable our customers to make informed decisions about which technologies and partners might best meet their own needs.
- **Foster a Collaborative Ecosystem:** Transparency about our use of partners' solutions encourages a collaborative environment where knowledge and best practices are shared, benefiting the entire ecosystem.

Benefits of Partner Integration

Integrating our partners into the core of our protocol offers several benefits:

- **Reliability and Security:** Using proven decentralized solutions enhances the reliability and security of our network, aligning with our goal to reduce reliance on centralized systems.
- **Scalability:** Leveraging partners' technologies allows us to scale more efficiently by utilizing their expertise and solutions to handle specific tasks or challenges.
- **Innovation:** This approach fosters innovation by encouraging the use of cutting-edge technologies and solutions that may not be available in traditional systems.

The strategic integration of our partners into the W3.io protocol is a deliberate choice that underscores our commitment to the decentralized model. It not only enhances our protocol's functionality and reliability but also sets a standard for transparency and collaboration in the web3 space. By doing so, we not only advocate for the adoption of decentralized solutions but also *lead by example*, showcasing the practical benefits and operational enhancements these technologies bring to the table.

6 User Experience (UX)

At W3.io, we simplify the complex landscape of web3 with a user experience designed to make hard things easy. Our interfaces guide both technical and non-technical users through the intricacies of blockchain technology, providing clear, actionable tools and resources. This approach not only exposes and demystifies potential dangers but also empowers our partners and customers to focus on solving their business challenges effectively. For developers, we streamline the foundational aspects of web3 development, allowing them to concentrate on creating innovative solutions rather than getting bogged down by routine tasks. The UI showcases how and why to use the tools and services offered by our partners, making it clear how each partner contributes to solving specific business problems. This visibility fosters an environment of innovation, where Customers and their developers are freed from the minutiae to focus on crafting new and unique solutions.

[DIAGRAM OF ON AND OFF-CHAIN COMPUTE INTEGRATION]

6.1 CLI

The Command Line Interface (CLI) provides power users with the ability to interact directly with the W3.io protocol through command-line tools. This interface is ideal for automation and scripting tasks.

6.2 Visual

The visual interface of W3.io is structured to facilitate the creation and management of recipes and ingredients, which are central to representing business problems within the w3.io protocol. This interface visually maps out the flow of operations, where each step or box in the flow chart represents an interaction with partners, blockchain technologies, or the recipe's data. This approach simplifies complex processes into manageable components, making it intuitive for users to design and oversee their business solutions.

7 Ingredients

Ingredients are the fundamental building blocks within the W3.io ecosystem, representing specific tasks or operations executable as part of a recipe. These modular and reusable components can execute functions such as accessing databases, performing calculations, or interacting with blockchains. They can add or modify variables within the recipe's variable pool to maintain state throughout execution and generate verifiable logs for integrity and traceability.

Ingredients can be simple, performing a single task, or compound, consisting of multiple sub-ingredients for more complex operations. This design enhances flexibility and scalability in developing business solutions.

A special type of Ingredient called a "Trigger" is the entry point to a recipe. Every business process can be considered a reaction to something, whether that is a schedule elapsing, an email receipt, an API call, or a blockchain event occurring. Anything like this can be built into a trigger, and when the trigger occurs, the recipe will run against the specific information from that trigger.

Partners will work with w3.io and other partners to ensure the ingredients integrate their tools well. Where partners naturally solve a more complex problem together, they can create and publish compound ingredients to the benefit of all involved. An example of that is below:

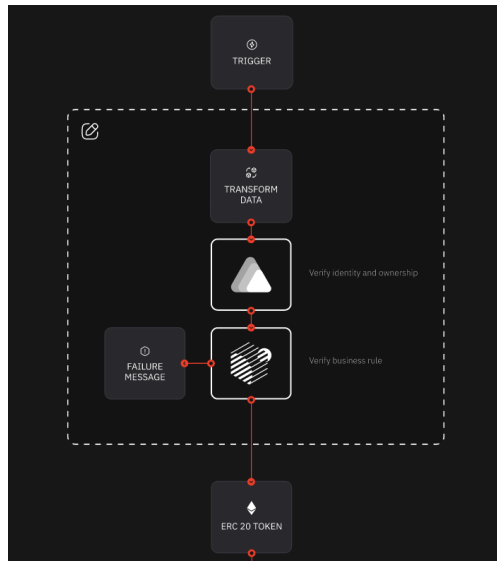


Figure 1: Illustration of a Compound Ingredient in W3.io

8 Recipes

Recipes in W3.io are structured workflows composed of multiple ingredients, defining complete business processes triggered by specific events. These can be considered the template for what should be done given a specific event defined by the trigger. When the trigger fires and the recipe begins to run, it's then running against specific conditions.

Each recipe includes:

- **Trigger:** Initiates execution, triggered by events such as a time-based event, a transaction on a blockchain, or an external API call.
- **Variable Pool:** Provides dynamic data storage, enabling ingredients to share and modify data throughout the recipe's execution.
- **Workflow Steps:** Ordered sequence where each ingredient processes data and passes results to the next step.
- **Failure Handling:** Mechanisms to manage errors or exceptions, including logging errors, retrying steps, or executing alternative workflows.

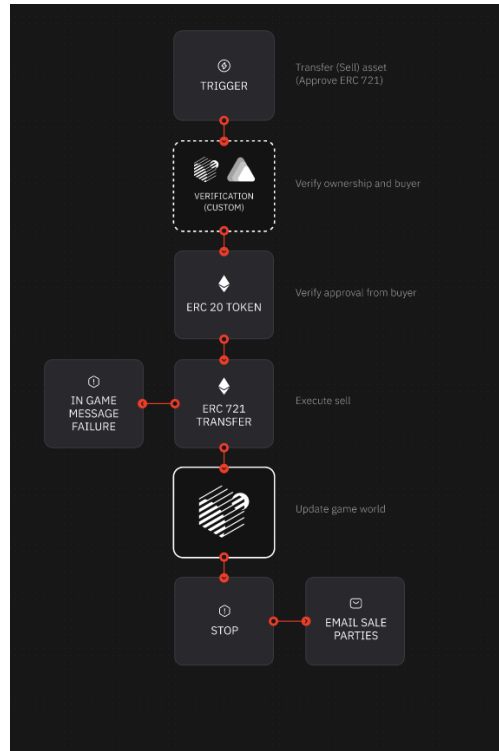


Figure 2: Illustration of a Typical Recipe Workflow in W3.io

Recipes are highly customizable and adaptable to a wide range of business needs, making them a powerful tool for integrating decentralized computing resources into coherent, executable workflows.

Deployment Once a recipe design is complete, then the customer would arrange payment for when the recipe runs, and then deploy the recipe into the w3.io protocol. Deployment tools within the visual interface allow users to easily deploy their recipes into the decentralized network, ensuring that their business processes are executed as designed.

Payment The payment interface simplifies the process of handling transactions within the W3.io network. It provides a unified method for managing payments for services rendered by various decentralized compute partners.

9 W3.io Protocol

The W3.io protocol serves as the foundational framework of the network, enabling the seamless integration of decentralized computing resources and blockchain technologies. At the heart of W3.io is

an incentivized network of nodes, each contributing computing power to the collective network. These nodes engage in a decentralized task queue that tracks and manages the execution of recipes and their constituent ingredients (or tasks). All nodes operate on a uniform core code, structured as a finite state machine, which collaborates across the network to achieve the necessary consensus for each recipe. Unlike traditional blockchain systems, these nodes operate at their native compute speeds, functioning purely as a distributed network of computers sharing computational resources. The next sections will get deeper into the network structure, node functionalities, and the decentralized task queue, providing a comprehensive understanding of the W3.io protocol's operational dynamics.

9.1 Network

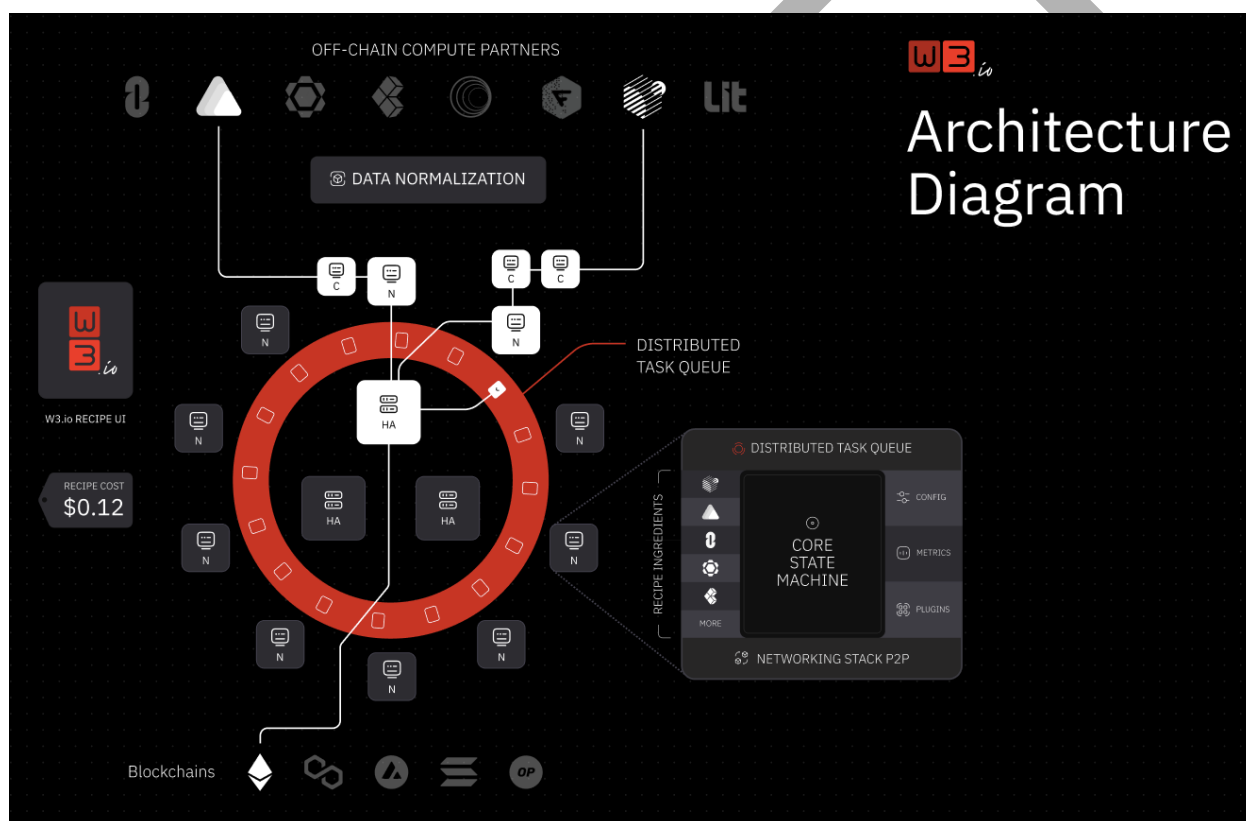


Figure 3: Architecture Overview of the W3.io Network

9.2 Nodes

Nodes are the operational units within the W3.io network, each contributing essential computing power to facilitate the execution of decentralized tasks. These nodes are incentivized to participate actively in the network, ensuring that customer-defined business processes are executed efficiently and reliably.

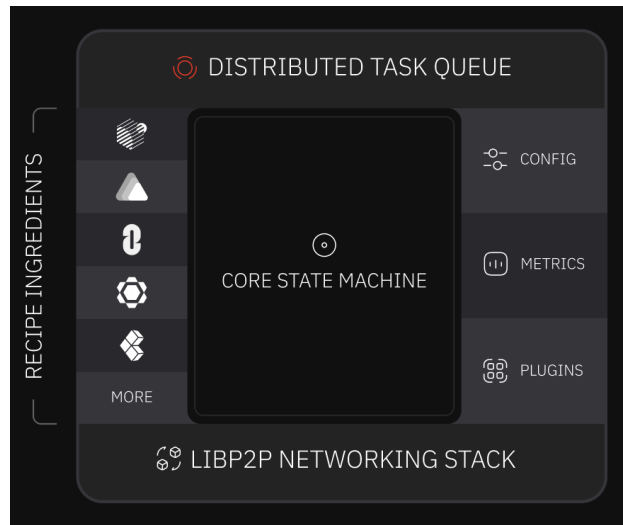


Figure 4: Node Architecture

Each node within the network operates based on a uniform core code, which functions as an event-driven finite state machine. This design is optimized to minimize the computational overhead not directly related to customer recipes, thereby enhancing the overall efficiency of the network.

The choice of programming language for implementing the nodes in the W3.io network was a critical decision influenced by several factors that prioritize safety, performance, and reliability. After careful consideration, Rust was selected as the primary language for node development. Rust provides several advantages that align well with the needs of a decentralized compute network like W3.io.

- **Memory Safety:** Rust's ownership model ensures memory safety without the overhead of garbage collection, which is crucial for maintaining high performance and avoiding common security vulnerabilities like buffer overflows (Matsakis & Klock, [2014](#)).
- **Concurrency:** Rust's approach to concurrency is based on the concept of 'fearless concurrency', which allows developers to write code that is free from data races and other concurrency issues, making it ideal for the distributed nature of the nodes (Levy, [2015](#)).
- **Performance:** Rust offers performance comparable to C and C++, which is essential for the compute-intensive tasks that nodes must handle in the W3.io network (Anderson, [2016](#)).
- **Robust Ecosystem:** Rust's growing ecosystem of libraries and tools, particularly for network programming and cryptography, provides a strong foundation for developing secure and efficient node software (Blandy & Orendorff, [2017](#)).

Core Components of a Node

- **LibP2P Networking Stack:** Each node is equipped with a LibP2P networking stack, which forms the backbone of node-to-node communication. This stack facilitates a robust and decentralized connectivity framework, allowing nodes to interact seamlessly across the network.
- **Core State Machine:** At the heart of each node is a core state machine that manages the node's operations based on incoming events. This state machine ensures that each node responds appropriately to changes in network conditions or task requirements.
- **Configuration System:** Nodes feature a flexible configuration system tailored to the hardware specifications on which they operate. This system allows nodes to optimize their performance and resource utilization effectively.
- **Metrics System:** A built-in metrics system enables the monitoring of each node's performance and participation in the network. This system is crucial for maintaining operational transparency and for the continuous improvement of node efficiency.
- **Plugins:** Nodes can be extended with various plugins that utilize the underlying hardware resources or add new functionalities. For example, a plugin might enable a node to register itself in a specific geographical locale or enhance its computational capabilities.
- **Recipe Ingredients:** Each node stores and manages the ingredients required to execute the recipes. These ingredients are modular components that nodes combine to perform complex business processes as defined in the recipes.
- **Distributed Task Queue:** Nodes participate in a distributed task queue, a crucial component that assigns tasks to nodes based on their availability and capability. This queue ensures that tasks are distributed efficiently across the network, minimizing delays and optimizing resource use.

These components collectively ensure that each node within the W3.io network can operate autonomously yet cohesively, contributing to the network's goal of providing a decentralized, efficient, and reliable platform for executing complex business workflows.

9.3 Types of Nodes

The W3.io network accommodates diverse computing resources by categorizing nodes into three types: High Availability Nodes, Normal Nodes, and Community Nodes. This classification is designed to maximize compute participation across different hardware capabilities and availability.

9.3.1 High Availability Nodes

High Availability Nodes are designed to offer robust services with a target of 99.999% uptime, often referred to as "five nines" availability. These nodes are typically equipped with advanced compute resources such as Trusted Execution Environments (TEEs), GPUs, substantial disk space, and high-performance CPUs. They are incentivized to maintain high availability and performance, playing a crucial role in handling critical tasks within the network.

9.3.2 Normal Nodes

Normal Nodes are akin to desktop computers in residential settings. They are expected to be online consistently, though they may occasionally face connectivity or power interruptions. These nodes form the backbone of the network, ensuring widespread participation and resource availability.

9.3.3 Community Nodes

Community Nodes include devices with intermittent connectivity and availability, such as laptops and smartphones. Users can connect these devices to the network during periods of non-use, such as overnight, to contribute resources while earning compute incentives. This category allows for casual participation from a broad user base, enhancing the network's decentralization.

Network Participation All nodes, regardless of their category, participate as consensus peers within the network. They execute tasks, or "ingredients," assigned to them, contributing to the network's overall computational power and reliability. Unlike the Internet Computer Protocol (ICP) by Dfinity, where providing compute resources can be a costly investment, W3.io aims to lower the barriers to entry. This approach allows individuals to contribute to the network with existing devices, reducing the need for specialized hardware and significant upfront costs. This model contrasts sharply with ICP's infrastructure requirements, making W3.io more accessible to a broader audience (Foundation, [2018](#)).

9.3.4 Compute Execution

Nodes handle various compute tasks, categorized into customer compute and protocol compute.

Customer Compute Customer compute tasks are initiated by external triggers. Nodes listen for these triggers and execute the corresponding recipes.

Protocol Compute Protocol compute tasks are internal operations necessary for the maintenance and operation of the W3.io network.

9.3.5 Verifiability

Verifiability in the W3.io network begins at the moment an operation is triggered. Multiple nodes may listen for and "hear" this trigger, which could originate from various sources such as API calls, scheduled events, or blockchain transactions. Upon detection, the data is immediately subjected to a verification process using a customer-defined consensus mechanism. This ensures the authenticity and correctness of the data before any further processing.

Once verified, the context—which includes details about the executing nodes, the incoming data, and recipe metadata—is recorded in an ephemeral log. This log entry is then hashed, and the resulting hash is passed along to the next step in the recipe execution process. This procedure of logging and hashing continues throughout the execution of the recipe.

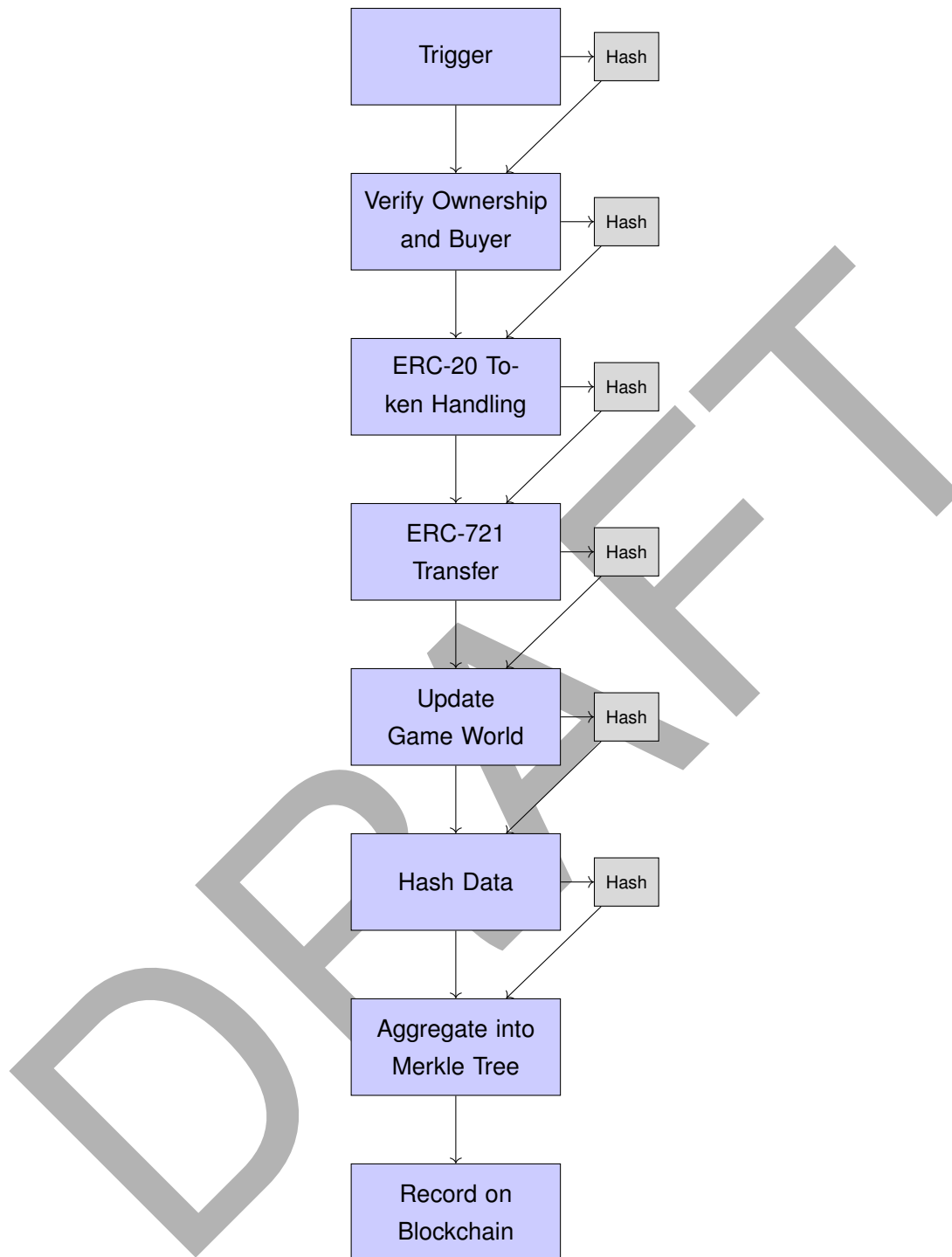


Figure 5: Verifiability Process in W3.io Network

Upon the completion of a recipe, the final hash, along with the hashes of other concurrently completing recipes, is aggregated into a Merkle tree. This tree is structured based on specific criteria such as time or the count of recipe runs. The root of this Merkle tree is then recorded on the customer's preferred

blockchain or on the default W3.io blockchain. This Merkel root serves as a verifiable record that can be used to confirm the integrity of any individual recipe execution.

Furthermore, the logged data is stored by default on IPFS (InterPlanetary File System), provided by Protocol Labs, one of our network partners. Customers also have the option to store their data on other long-term storage solutions such as Arweave or Space & Time, depending on their specific requirements.

This comprehensive approach to verifiability not only ensures that each recipe runs as intended but also allows customers to verify the results for as long as the logs are maintained. For perpetual verification, customers can opt for permanent storage solutions. The trust assumptions of our partners are also considered and integrated into the metadata of each operation, ensuring that all aspects of the execution are transparent and verifiable.

9.3.6 Consensus

The W3.io network employs a flexible consensus mechanism, allowing for the adaptation of various consensus algorithms based on the specific needs of each recipe. This flexibility is crucial as it enables the network to operate efficiently without being bound to a single consensus method, which can often be costly and less adaptable.

We support several notable consensus mechanisms, each chosen for its unique benefits depending on the operational requirements:

- **Tendermint:** Combines the speed of consensus algorithms like PBFT with the decentralized nature of blockchain. It is well-suited for blockchain services that require high performance and fast finality. Tendermint is particularly effective in environments where participants are known and reliability is required (Kwon, [2014](#)).
- **Practical Byzantine Fault Tolerance (PBFT):** Designed to work well in asynchronous systems, PBFT can provide high throughput and low latency, making it ideal for distributed networks that need to achieve consensus even in the presence of faulty or malicious nodes (Castro & Liskov, [1999](#)).
- **Hedera Hashgraph:** Known for its high throughput and fast finality, Hedera is ideal for applications requiring quick transaction processing (Baird, [2016](#)).
- **Stellar Consensus Protocol (SCP):** Offers low latency and flexible trust, suitable for low-cost and efficient consensus without requiring intensive proof-of-work (Mazières, [2015](#)).

Nodes participating in a recipe's consensus are selected by the decentralized task queue and work

collaboratively to reach consensus at each step of the recipe. This includes both read and write operations:

- **Write Operations:** Require consensus before execution; the decentralized task queue then directs the write to an appropriate executor with the signed consensus of the nodes.
- **Read Operations:** All reads must be validated by consensus. Special considerations are made for operations like proof of inference, where reads may not be identical but still need to be verified.

The choice of consensus mechanism can significantly impact the performance and cost-effectiveness of a recipe. For example, a cost-sensitive customer might opt for Stellar's SCP for its efficiency, while a security-focused customer may prefer Ethereum's robust Proof of Work system. Ultimately, W3.io's flexible consensus approach ensures that the results of any recipe or ingredient run are verifiably correct and then verifiably recorded.

9.3.7 Trusted Execution Environments (TEEs)

Trusted Execution Environments (TEEs) are utilized within certain nodes of the W3.io network to enhance security and provide fully automated transaction signing. These environments provide an isolated execution space that protects sensitive computations from external threats, including those from the host machine. Nodes equipped with TEEs are incentivized to perform sensitive or critical tasks, ensuring robust security measures are in place for operations that require higher trust levels.

We collaborate with partners like Cubist, Lit Protocol, etc, which specialize in providing solutions along these lines. These partnerships are part of our roadmap towards a fully decentralized offering. Over time, we aim to integrate more decentralized TEE solutions to align with our vision of a fully decentralized network capable of permissioned execution on our customers behalf within their complex business processes. This strategic approach allows us to leverage advanced security technologies while working towards greater decentralization.

9.4 Protocol Services

[AUDIE TO WRITE]

9.4.1 API

[AUDIE TO WRITE]

9.4.2 UI

[AUDIE TO WRITE]

9.4.3 Decentralized Task Queue

[AUDIE TO WRITE]

10 Token Model (Light Overview)

[BEN AND AUDIE TO WRITE]

11 Business Model

[DAVID AND AUDIE TO WRITE]

12 Conclusion

The conclusion

References

- Anderson, B. (2016). What is rust and why is it so popular? <https://stackoverflow.blog/2020/01/20/what-is-rust-and-why-is-it-so-popular/>
- Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM*, 53(4), 50–58. <https://dl.acm.org/doi/10.1145/1721654.1721672>
- Baird, L. (2016). The swirls hashgraph consensus algorithm: Fair, fast, byzantine fault tolerance. <https://www.swirls.com/downloads/SWIRLDS-TR-2016-01.pdf>
- Blandy, J., & Orendorff, J. (2017). *Programming rust: Fast, safe systems development*. O'Reilly Media. <https://www.oreilly.com/library/view/programming-rust/9781491927274/>
- Bonomi, F., Milito, R., Zhu, J., & Addepalli, S. (2012). Fog computing and its role in the internet of things. *Proceedings of the first edition of the MCC workshop on Mobile cloud computing*. <https://dl.acm.org/doi/10.1145/2342509.2342513>
- Castro, M., & Liskov, B. (1999). Practical byzantine fault tolerance. *Proceedings of the Third Symposium on Operating Systems Design and Implementation*. <https://pmg.csail.mit.edu/papers/osdi99.pdf>
- Elliot, S. (2014). Devops and the cost of downtime: Fortune 1000 best practice metrics quantified. <https://www.idc.com/getdoc.jsp?containerId=259972>
- Foundation, D. (2018). Dfinity technology overview series, consensus system. <https://dfinity.org/pdf-viewer/pdfs/viewer?file=../library/dfinity-consensus.pdf>
- Kwon, J. (2014). Tendermint: Consensus without mining. <https://tendermint.com/static/docs/tendermint.pdf>
- Lessig, L. (1999). *Code and other laws of cyberspace*. Basic Books. <https://archive.org/details/codeotherlawsofc0000less/page/n9/mode/2up>
- Levy, B. (2015). Ownership is theft: Experiences building an embedded os in rust. *Proceedings of the 8th Workshop on Programming Languages and Operating Systems*. <https://dl.acm.org/doi/10.1145/2818302.2818306>
- Matsakis, N., & Klock, F. (2014). The rust language. *ACM SIGAda Ada Letters*, 34(3), 103–104. https://ia903204.us.archive.org/view_archive.php?archive=/20/items/acm-digital-library-2020_20200719/fulltext/155.zip&file=10.1145%2F2663171.2663188.pdf
- Mazières, D. (2015). The stellar consensus protocol: A federated model for internet-level consensus. <https://www.stellar.org/papers/stellar-consensus-protocol.pdf>
- Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>
- Vaquero, L. M., & Roderio-Merino, L. (2014). Finding your way in the fog: Towards a comprehensive definition of fog computing. *ACM SIGCOMM Computer Communication Review*, 44(5). <https://dl.acm.org/doi/10.1145/2677046.2677052>
- Zyskind, G., Nathan, O., & Pentland, A. (2015). Decentralizing privacy: Using blockchain to protect personal data. *Proceedings of the IEEE Security and Privacy Workshops*. <https://ieeexplore.ieee.org/document/7163021>

A Network Partners



Figure 6: Network Partners

[GRAPHIC DESIGNER TO FIX]

B Team Profiles

[GRAPHIC DESIGNER AND MARKETING]

DRAFT

C Appendix C: Mathematical Models

Mathematical Model for Task Distribution in a Decentralized Network

In this section, we develop a mathematical model to describe the distribution of tasks across a decentralized network. The model aims to ensure a fair and efficient allocation of tasks to various nodes within the network, leveraging cryptographic hash functions to achieve randomness and uniformity in task assignment. This approach not only enhances the security of the task distribution process but also ensures that no single node is overloaded, thereby maintaining the network's performance and reliability.

The model is designed to be scalable and adaptable, capable of handling varying numbers of nodes and tasks without significant changes to the underlying algorithm. This flexibility is crucial for the dynamic environment of decentralized networks, where the number of participating nodes and the volume of tasks can fluctuate significantly.

By mathematically modeling the task distribution, we provide a robust framework that supports the decentralized nature of the network and aligns with the principles of fairness and efficiency that are critical to the success of decentralized systems.

Variables

- N : Total number of nodes in the network.
- R : Total number of recipes that need to be executed.
- C : Consensus group size required for each recipe.
- i : Index of the current recipe.
- S_i : Set of nodes selected for recipe i .
- $\text{hash}(x)$: A cryptographic hash function.

Process

1. Initial Selection:

$$S_1 = \{\text{hash}(1) \bmod N, \text{hash}(2) \bmod N, \dots, \text{hash}(C) \bmod N\}$$

Ensure uniqueness in S_1 by rehashing if necessary.

2. Subsequent Selections:

$$S_i = \{\text{hash}(S_{i-1}[1] \oplus i) \bmod N, \dots, \text{hash}(S_{i-1}[C] \oplus i) \bmod N\}$$

$\oplus$ denotes a bitwise XOR operation.

3. Validation:

$$\text{hash}(\text{hash}(S_{i-1}[j] \oplus i) \oplus j) \bmod N$$

Check for node repetition and recompute if necessary.

Justification

- **Uniformity:** The hash function ensures uniform distribution of node indices.
- **Memory:** Incorporating S_{i-1} prevents the same nodes from being selected repeatedly.
- **Scalability:** The method scales linearly with the number of nodes and recipes.